LAB6

Data Structure

Lab6

- The deadline for lab6 submission is 9th April at 11:59 pm.
- If you have any questions, please contact TA through email. (112alsdud@naver.com)
- Lab scores will be uploaded to the LMS next week.
- Questions about Lab 6 scores will only be accepted during next lab session.

- Code name: {student_id}_p6.c
- The code will be tested with 5 different input files.
- Each input file will be worth 20 points
- Make sure that the output format matches the provided example exactly.

Evaluation criteria

Category	Evaluation	
p6	100	
Total	100	

- *Use GCC **13.3** version.*
- *No score will be given if there is a compile error caused by difference of gcc version*

Lab6 – Complete binary tree

- Design complete binary tree ADT by using array.
- Use complete binary tree ADT to perform pre-, in-, and post-order traversal using recursive functions.
- In addition to the traversals, your program must also:
  - Count the number of leaf nodes in the tree using recursive functions.
  - Compute the height of the tree using recursive functions (leaf nodes are at height 0).
  - Determine whether the tree is a perfect binary tree.

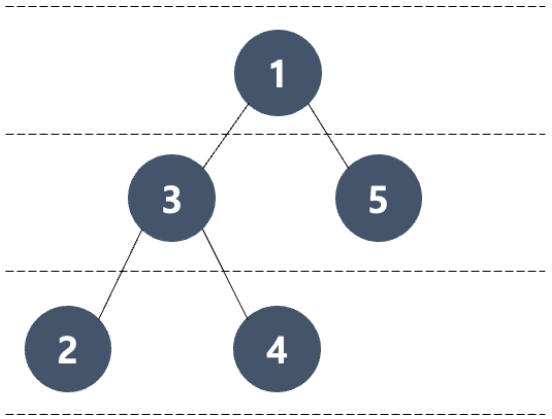
```
5
1 3 5 2 4
```

Input.txt

Complete Binary Tree ADT

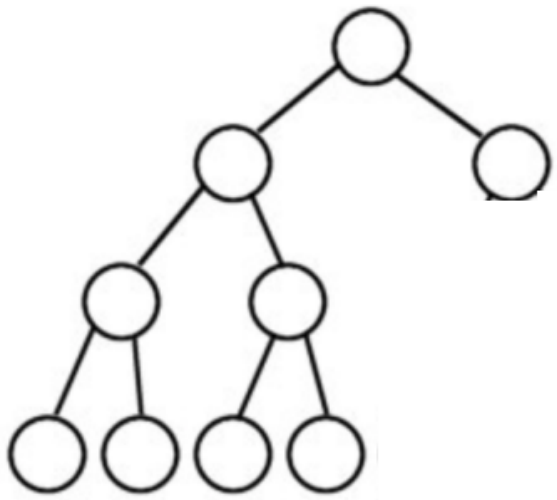
```
Preorder: 1 3 2 4 5
Inorder: 2 3 4 1 5
Postorder: 2 4 3 5 1
Leaf Nodes: 3
Tree Height: 2
Is Perfect Binary Tree: No
```

Output.txt

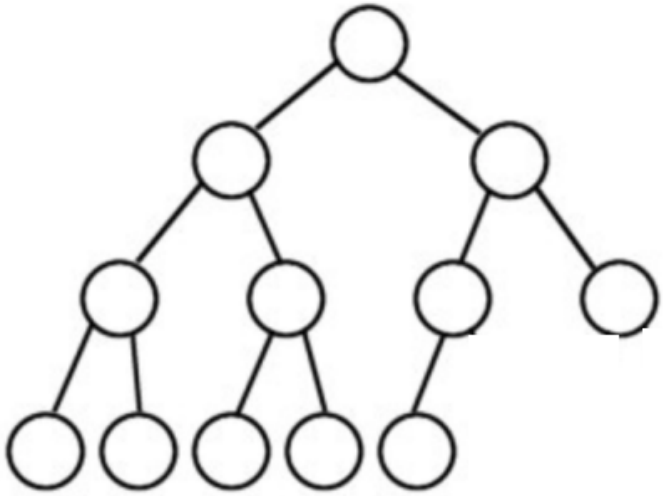


Lab6 – Complete binary tree

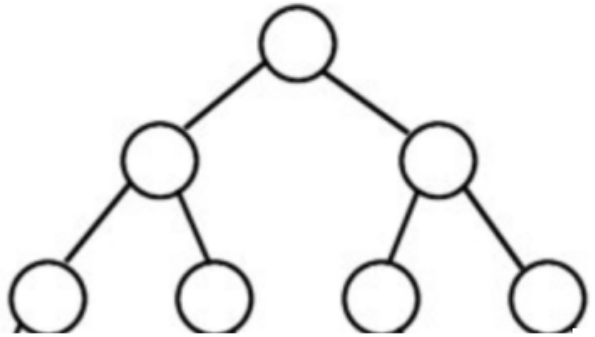
- Full binary tree (전 이진트리) is a binary tree in which every node has 0 or 2 children
- Complete binary tree (완전 이진트리) is a binary tree in which every level, except the last, is completely filled and the last level has all its nodes to the left side
- Perfect binary tree (포화 이진트리) is a binary tree in which all interior nodes have two children and all leaves have the same depth or same level



full binary tree



complete binary tree



perfect binary tree

Lab6 – Complete binary tree

- Structure

```
struct TreeStruct{
        int size;
        int numOfNode;
        int* element;
};
typedef struct TreeStruct * Tree;
```

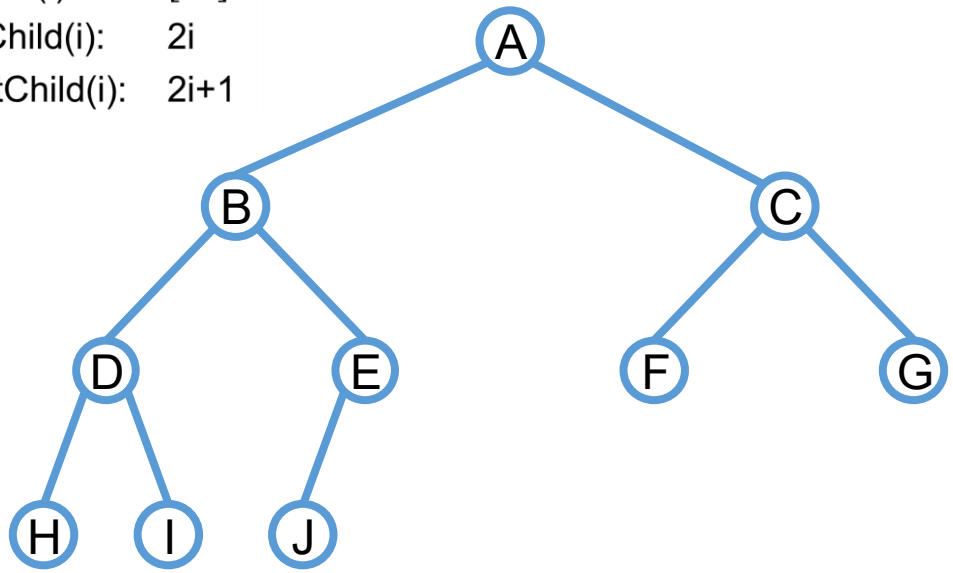
- Complete binary tree

i is index

parent(i): ⌊i/2⌋

leftChild(i): 2i

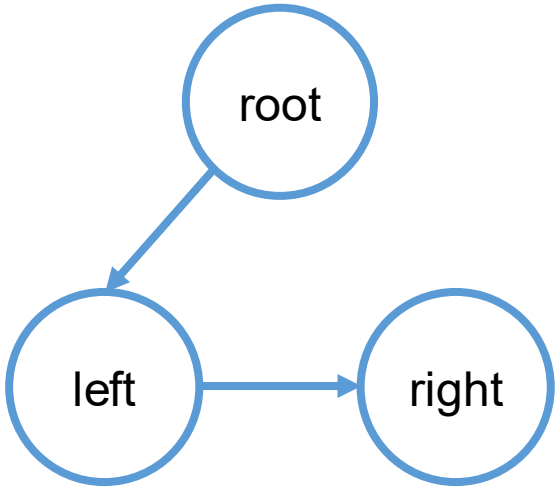
rightChild(i): 2i+1



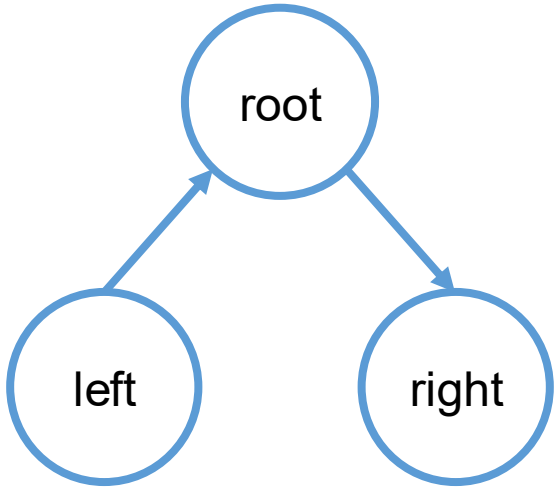
	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]	[10]
	A	B	C	D	E	F	G	H	I	J

Lab6 – Complete binary tree

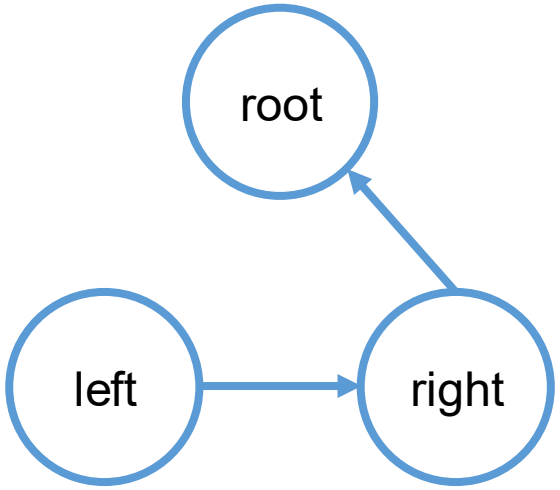
- Preorder



- Inorder



- Postorder



Lab6 – Complete binary tree

- Structure

```
struct TreeStruct{
        int size;
        int numOfNode;
        int* element;
};
typedef struct TreeStruct * Tree;
```

- Function

```
<Lab6>
Tree CreateTree(int size);
void Insert(Tree tree, int value);
void printPreorder(Tree tree, int index);
void printInorder(Tree tree, int index);
void printPostorder(Tree tree, int index);
int CountLeafNodes(Tree tree, int index);
int ComputeHeight(Tree tree, int index);
int IsPerfectTree(Tree tree);
void printTree(Tree tree);
void DeleteTree(Tree tree);
```

Lab6 – Complete binary tree

- **Tree CreateTree(int size)**
  - Allocates memory for a complete binary tree using an array. Initializes the tree with a specified maximum size.
- **void Insert(Tree tree, int value)**
  - Insert a new node in the tree.
  - If tree is full, just print an error message : 'Error! Tree is full.'
- **void PrintInorder**
  - Print the tree by inorder traversal using recursive function.
- **void PrintPreorder**
  - Print the tree by preorder traversal using recursive function.
- **void PrintPostorder**
  - Print the tree by postorder traversal using recursive function.

Lab6 – Complete binary tree

- **int CountLeafNodes(Tree tree, int index);**
  - Count the number of leaf nodes in the tree using recursive function and return it.
- **int ComputeHeight(Tree tree, int index);**
  - Compute the height of the tree using recursive function and return it.
  - The height of leaf nodes is 0.
- **int IsPerfectTree(Tree tree);**
  - Check whether the tree is a perfect binary tree or not.
- **void printTree(Tree tree);**
  - Display all traversals (pre-, in-, post-) and tree properties (leaf count, height, and whether it is a perfect binary tree)
- **void DeleteTree(Tree tree);**
  - Free all dynamically allocated memory for the tree.

Lab6 – Complete binary tree

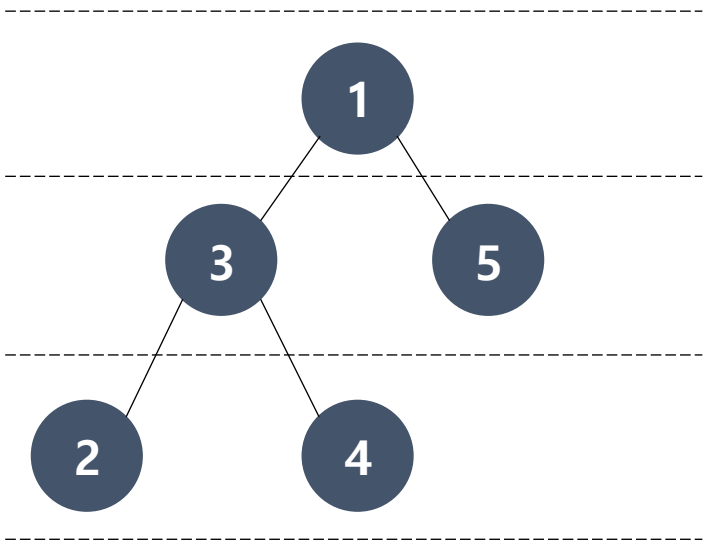
```c
#include<stdio.h>
#include<stdlib.h>

struct TreeStruct{
        int size;
        int numOfNode;
        int* element;
};

typedef struct TreeStruct* Tree;

Tree CreateTree(int size);
void Insert(Tree tree, int value);
void PrintTree(Tree tree);
void PrintPreorder(Tree tree, int index);
void PrintInorder(Tree tree, int index);
void PrintPostorder(Tree tree, int index);
int CountLeafNodes(Tree tree, int index);
int ComputeHeight(Tree tree, int index);
int IsPerfectTree(Tree tree);
void DeleteTree(Tree tree);
```

```c
void main(int argc, char* argv[])
{
        FILE *fi;
        Tree tree;
        int treeSize;
        int tmpNum;

        fi = fopen(argv[1], "r");
        fscanf(fi, "%d", &treeSize);
        tree = CreateTree(treeSize);

        while(fscanf(fi, "%d", &tmpNum) == 1)
        {
                Insert(tree, tmpNum);
        }
        PrintTree(tree);
        DeleteTree(tree);
}
```

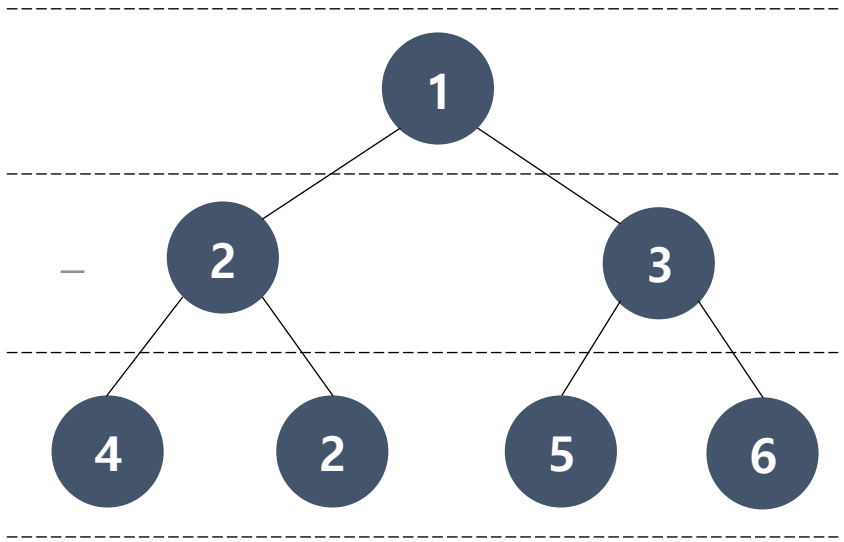
Lab6 – Complete binary tree

- Example 1)

```
~/Dropbox/연구실/자료구조/2026자료구조/실습/Lab6/code ❯ cat input_ex1.txt
5
1 3 5 2 4
~/Dropbox/연구실/자료구조/2026자료구조/실습/Lab6/code ❯ ./p6 input_ex1.txt
Preorder: 1 3 2 4 5
Inorder: 2 3 4 1 5
Postorder: 2 4 3 5 1
Leaf Nodes: 3
Tree Height: 2
Is Perfect Binary Tree: No
```



Lab6 – Complete binary tree

- Example)

```
[~/Dropbox/연구실/자료구조/2026자료구조/실습/Lab6/code ❯ cat input_ex2.txt
7
1 2 3 4 2 5 6 8 9
[~/Dropbox/연구실/자료구조/2026자료구조/실습/Lab6/code ❯ ./p6 input_ex2.txt
Error! Tree is full.
Error! Tree is full.
Preorder: 1 2 4 2 3 5 6
Inorder: 4 2 2 1 5 3 6
Postorder: 4 2 2 5 6 3 1
Leaf Nodes: 4
Tree Height: 2
Is Perfect Binary Tree: Yes
```